

AERIS DEVELOPMENT — CHECKPOINT 3 REPORT This chat completed the transition from single-geometry generation to parametric design-space sampling. The project now has: a clean CLI architecture a modular geometry engine reproducible run pipelines Latin Hypercube Sampling integration This is now a real dataset-generation platform, not a research script.

1 — What Was Built Before This Chat Earlier phases built the software infrastructure. These components already existed.

CLI System The CLI is defined here: `src/aeris/cli.py` The CLI exposes command groups. Example: `aeris geometry generate aeris pipeline smoke aeris version` Implementation reference: Typer is used so the system can grow into a multi-command platform.

Run Management System Every execution creates a reproducible run folder. Defined in: `common/paths.py` Run structure: `data/runs/` `2026-03-12_194500_geometry_wing_bwb/` `logs/` `artifacts/` `manifest.json` `input_config.yaml` Reference implementation: This guarantees: reproducibility experiment traceability OS-agnostic structure

Logging System Logging goes to both console and file. `common/logging_utils.py` Reference: `YAML Config Loader` Configs define the entire geometry generator. `configs/geometry/wing_bwb.yaml` Loader implementation:

2 — Geometry Engine Architecture The geometry engine lives here: `src/aeris/geometry/` Core modules: `params.py` `sampling.py` `planform.py` `sections.py` `validation.py` `export.py` `plotting.py` `aerosandbox_adapter.py` Each module has a single responsibility.

3 — Geometry Parameter System The configuration schema is defined here: `geometry/params.py` The core data classes: `BWBGeneratorConfig` `BWBDesignSample` These represent: configuration sampled design variables Reference: 17 Independent Design Variables The design vector: `c1_m` `c2_ratio` `c3_ratio` `c4_ratio` `b_total_m` `b3_ratio` `split_ratio` `sw1_deg` `sw2_deg` `sw3_deg` `twist_b0_deg` `twist_b1_deg` `twist_b2_deg` `twist_b3_deg` `dihedral_b1_deg` `dihedral_b2_deg` `dihedral_b3_deg` These form the design space.

4 — Random Sampling System Before LHS, sampling used pure RNG. File: `geometry/sampling.py` Reference: This samples each variable within YAML bounds.

5 — Planform Generator The core aerodynamic geometry generator. File: `geometry/planform.py` Reference: Responsibilities: compute chord distribution generate leading edge curve create spline + linear trailing edge compute planform metrics discretize span Outputs: `PlanformResult`

6 — Section Geometry Builder File: `geometry/sections.py` Reference: Responsibilities: build spanwise sections interpolate twist interpolate dihedral generate 3D section records Output: `SectionGeometryResult`

7 — AeroSandbox Integration The geometry is converted into an AeroSandbox airplane model. File: `geometry/aerosandbox_adapter.py` Reference: This produces: `asb.Wing` `asb.Airplane` `aspect_ratio`

8 — Geometry Validation System File: `geometry/validation.py` Reference: Validations include: chord > 0 monotonic span finite geometry proper discretization This ensures dataset quality.

9 — Geometry Export System Artifacts exported include: `control_points.csv` `planform_sections.csv` `section_3d.csv`

geometry_summary.json planform.png Implementation: geometry/export.py
Reference: These outputs are ML-ready geometry datasets. 10 — Geometry Pipeline
The entire generation process runs through: pipeline/geometry_run.py Reference:
Pipeline steps: load YAML config validate config initialize RNG sample design
generate planform build sections build AeroSandbox model validate geometry export
artifacts write manifest 11 — LHS Sampling (NEW IN THIS CHAT) This chat
implemented Latin Hypercube Sampling. File: geometry/lhs.py Reference:
Important function: generate_lhs_samples(...) Process: 1 generate unit hypercube 2
stratify each dimension 3 scale to variable bounds 4 convert matrix ->
BWBDesignSample objects This ensures: uniform design space coverage no
clustering better surrogate datasets 12 — LHS → Geometry Test The script:
scripts/test_lhs_to_geometry.py Tested the pipeline: LHS sample → planform
generator → section generator → AeroSandbox model Result: Success: LHS sample
was converted into a valid geometry. Example metrics: span $\approx$ 3.89 m area $\approx$ 2.42 m²
AR $\approx$ 6.29 sections = 99 This confirms: The LHS sampling and geometry engine are
fully compatible. 13 — Important Architectural Decisions Made Seed usage Seed is
fixed globally. Purpose: control internal perturbations segment lengths sweep
perturbations Not the main design variables. LHS variables LHS samples only the 17
true design variables. These define the global design space. Perturbations These
produce local geometry realism. Examples: segment_length_variation
sweep_variation curvature strength Controlled by seed. 14 — Current System Status
The system now supports: config driven generation reproducible runs planform
generator section generator AeroSandbox integration validation system artifact
export LHS sampling This means: The platform is ready to move from: single
geometry generation to large dataset generation 15 — The Next Phase Next phase:
DATASET GENERATION ENGINE Goal: Generate thousands of geometries
automatically. Example: aeris dataset generate --config wing_bwb.yaml --n 5000 This
phase requires: 1 dataset CLI command 2 dataset directory system 3 metadata
export 4 generator versioning 5 LHS integration into pipeline